

DELIVERABLE 2: SYSTEM DESIGN DOCUMENT

Smart Asset Management and Resource Allocation Platform

FULL-STACK ARCHITECTURAL SPECIFICATION & DATA DESIGN LEDGER

TARGETED FOR THE CULTURAL COUNCIL INFRASTRUCTURE OF IIT ROORKEE

Prepared by:

RISHIT YADAV

DEPARTMENT OF CHEMICAL ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

COMPILED ON: JUNE 8, 2026

Contents

1	Problem Understanding	2
2	System Architecture	2
2.1	Presentation Tier (Frontend Client App)	2
2.2	Application Tier (Backend API Framework)	2
2.3	Data Storage Tier (Database Engine)	2
3	Database Schema	3
3.1	The <code>users</code> Table	3
3.2	The <code>assets</code> Table	3
3.3	The <code>bookings</code> Table	3
4	Entity Relationship Diagram (ERD)	4
5	API Overview	4
6	Design Decisions	5
6.1	Database-Level Inventory Triggers vs. Application Logic	5
6.2	Stateless JWT Verification with Property Naming Mismatches	5

1 Problem Understanding

The Cultural Council of IIT Roorkee acts as a centralized administrative node overseeing a diverse pool of shared, operational infrastructure assets across multiple active technical sections and clubs. These high-value assets encompass digital media tools (DSLR cameras, recording gear), specialized event production items (studio lighting setups, audio arrays), and presentation materials (costumes, stage props, event infrastructure).

Managing allocations across numerous simultaneous campus events introduces severe logistical complexities. The historical reliance on fragmented channels, manual offline paper registers, and individual spreadsheets presents major operational limitations:

- **Scheduling Conflicts:** Zero visibility into upcoming reservations leads directly to double-bookings and resource bottlenecks.
- **Underutilization:** Lack of an accessible inventory tracking catalog leaves available assets forgotten during event preparation windows.
- **Accountability Blind Spots:** Manual records complicate tracking item status, resulting in late returns, unrecorded damages, and missing historical metrics.

This application addresses these operational challenges by introducing a centralized full-stack ecosystem. The platform enforces secure role-based access controls, dynamic availability filtering, atomic checkout constraints, and transparent user dashboard history matrices to preserve absolute tracking consistency throughout the asset lifecycle.

2 System Architecture

The system layout uses a highly decoupled, modular 3-Tier Architectural Topology to maintain clean data boundary separation, facilitate testing, and ensure cross-platform reproducibility:

2.1 Presentation Tier (Frontend Client App)

The presentation tier is engineered as a component-driven Single Page Application (SPA) utilizing React.js, running inside the independent `/frontend` workspace directory. Styling layouts are declared natively via Tailwind CSS utility classes to achieve an intuitive, fluid user experience across both desktop viewports and mobile devices. Client components manage local component rendering scopes (such as searching, filtering parameters, and reservation forms) and make asynchronous XMLHttpRequests to the server endpoints.

2.2 Application Tier (Backend API Framework)

The application server tier uses Node.js running an Express.js routing infrastructure directly from the repository's root workspace folder. This framework parses raw incoming operations, checks permissions, and communicates with the underlying storage layer. Session states are maintained stateless using JSON Web Token (JWT) verification filters. A custom interceptor middleware splits client authorization headers to validate signed payload contents, safely injecting core user parameters (`userId`, `role`) into incoming request chains.

2.3 Data Storage Tier (Database Engine)

The core storage mechanism is managed via a relational PostgreSQL database management instance. Connection tasks are handled through an efficient pooled client connection driver array managed in `db.js`. Transaction boundaries are explicitly defined to isolate simultaneous multi-user interactions, protecting item counts from data corruption or asynchronous overwrites.

3 Database Schema

The database layout relies on structured table definitions to organize properties safely, applying primary keys, unique checks, and cascade conditions to prevent data leaks or orphan records:

3.1 The users Table

Stores profile metrics, credentials, and access roles.

- `id` (SERIAL): Primary Key. Unique internal autoincrement index.
- `name` (VARCHAR(255)): Standard name identity string.
- `email` (VARCHAR(255)): Unique constraint. Secure login address token.
- `password` (VARCHAR(255)): Stores cryptographically hashed strings.
- `role` (VARCHAR(50)): Checked constraint limiting scopes exclusively between 'admin' and 'consumer'.

3.2 The assets Table

Indexes available equipment, description strings, and available unit levels.

- `id` (SERIAL): Primary Key. Relational identifier token.
- `name` (VARCHAR(255)): The structural display name of the inventory asset.
- `category` (VARCHAR(100)): Group mapping index across council divisions.
- `description` (TEXT): Outlines operational boundaries and specifications.
- `quantity_available` (INT): Enforces a strict non-negative check limitation (≥ 0).

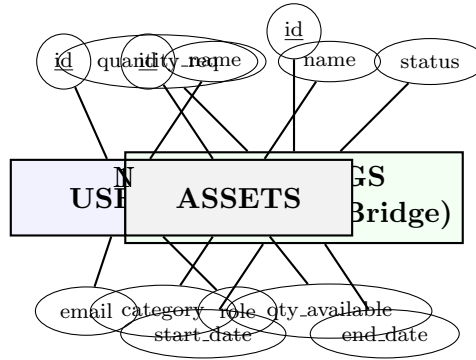
3.3 The bookings Table

Tracks physical reservation lines, durations, and state transitions.

- `id` (SERIAL): Primary Key. Relational booking tracker key.
- `useriid(INT)` : *ForeignKeylinkingto*users(id)*with*ON DELETE CASCADE.
- `assetiid(INT)` : *ForeignKeylinkingto*assets(id)*with*ON DELETE CASCADE.
- `quantity_requested` (INT): Checked validation rule forcing value limits (> 0).
- `start_date / end_date` (TIMESTAMP): Records the duration window.
- `status` (VARCHAR(50)): Checked filter tracking values ('pending', 'approved', 'rejected').

4 Entity Relationship Diagram (ERD)

The relational topology of the physical database layer maps entities across clear structural bounds. The primary `bookings` intersection mapping entity functions as a many-to-one bridge linking back to both individual user profile identities and the main equipment repository indexes:



5 API Overview

Decoupled boundaries securely pass application state variables via structured REST API endpoints. Session validation layers are handled through standard token extraction logic matching matching context paths:

Table 1: System Endpoint Routing Specifications

HTTP Method	API Endpoint Path	Operational Scope / Target Lifecycle
POST	/api/auth/register	Generates fresh unique account record schemas.
POST	/api/auth/login	Verifies passwords to issue signed JWT strings.
GET	/api/assets	Dynamic search partial query parameter extraction.
POST	/api/bookings/request	Logs reservation times into database tracks.
GET	/api/bookings/my-history	Compiles personal logs via optimized user id joins.
PATCH	/api/bookings/:id/approve	Admin workspace action executing automated trigger.
PATCH	/api/bookings/:id/reject	Admin workspace action setting status to rejected.

6 Design Decisions

6.1 Database-Level Inventory Triggers vs. Application Logic

A core structural design decision was to manage real-time inventory deductions directly at the database storage layer using a native PL/pgSQL procedural trigger, rather than standard backend application scripts.

Processing calculation changes inside standard API route functions introduces dangerous timing windows and race conditions. For instance, if multiple users check out a single remaining camera at the exact same millisecond, script-level checks can misread stock states, resulting in system over-allocation errors.

By offloading this logic to a database trigger, item tracking remains atomic and completely fail-safe. The database blocks conflicting operations instantly, safeguarding inventory tracking integrity during periods of high user activity:

```
1 CREATE OR REPLACE FUNCTION adjust_asset_stock_on_approval()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     -- Atomic execution handles deduction exactly when status
4     transitions to approved
5     IF NEW.status = 'approved' AND (OLD.status = 'pending') THEN
6         UPDATE assets
7         SET quantity_available = quantity_available - NEW.
            quantity_requested
8         WHERE id = NEW.asset_id;
9     END IF;
10    RETURN NEW;
11 END;
12 $$ LANGUAGE plpgsql;
13
14 CREATE TRIGGER trg_adjust_asset_stock
15 AFTER UPDATE OF status ON bookings
16 FOR EACH ROW
17 EXECUTE FUNCTION adjust_asset_stock_on_approval();
```

Listing 1: PostgreSQL Automated Calculation Trigger Strategy

6.2 Stateless JWT Verification with Property Naming Mismatches

The application layer uses stateless JSON Web Token (JWT) tracking to enforce reliable role-based routing security without adding memory overhead to the server instance. During frontend integration testing, we resolved a subtle payload naming conflict between the signing payload definition and backend execution parameters. While the authentication middleware initialized tracking objects under the custom variable property `req.user.userId`, subsequent resource routes expected a standard parameter key (`req.user.id`). This variation broke relational table integrity by inserting null rows into database allocations.

To resolve this and make the system robust against multiple tracking setups, a fallback check configuration (`const user_id = req.user.id || req.user.userId;`) was implemented across all routes, stabilizing session lookups.